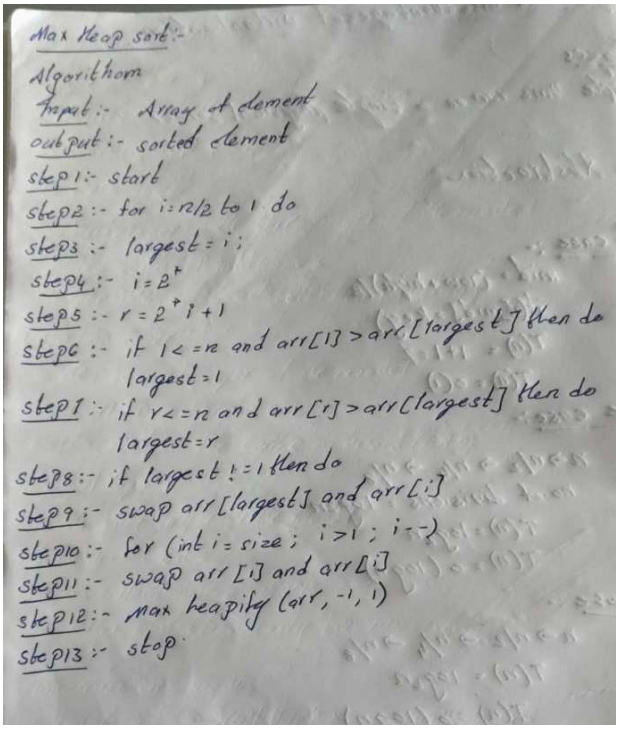
PRACTICAL: 03

AIM :- Implementation of max-heap sort algorithm

Algorithms:-

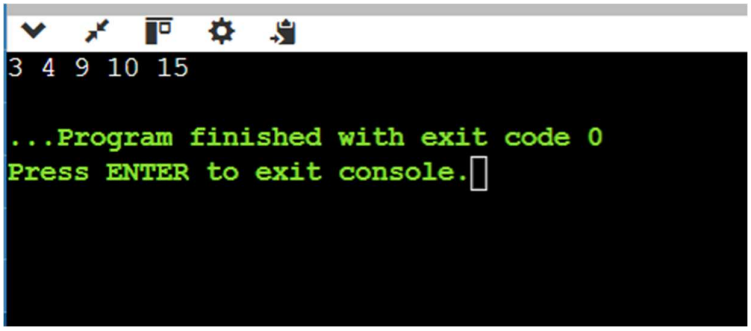


Program/code:-

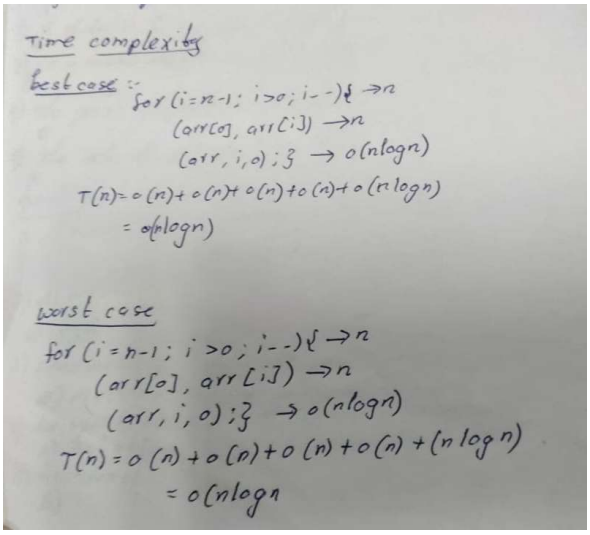
```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
 int largest = i;
 int left = 2 * i + 1;
 int right = 2 * i + 2;
 if (left < n && arr[left] > arr[largest]) {
```

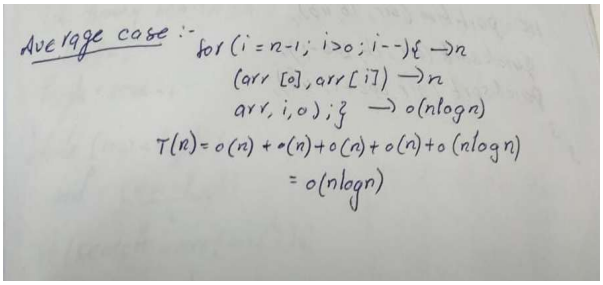
```
 largest = left;
 }
 if (right < n && arr[right] > arr[largest]) {
 largest = right;
 }
 if (largest != i) {
 swap(arr[i], arr[largest]);
 heapify(arr, n, largest);
 }
}
void heapSort(int arr[], int n) {
 for (int i = n / 2 - 1; i >= 0; i--) {
 heapify(arr, n, i);
 }
 for (int i = n - 1; i > 0; i--) {
 swap(arr[0], arr[i]);
 heapify(arr, i, 0);
 }
}
int main() {
 int arr[] = {4, 10, 3, 5, 1};
 int n = 5;
 heapSort(arr, n);
 for (int i = 0; i < n; i++) {
 cout << arr[i] << " ";
 }
 return 0;
}
```

Output:-



Timecomplexity:-





Conclusion:-

In conclusion, Max Heap Sort is an efficient comparison-based sorting algorithm that uses a max heap data structure to arrange elements in ascending order. It has a time complexity of O(n log n) in the best, average, and worst cases, making its performance predictable. It also requires only O(1) extra space when implemented in-place. Therefore, Max Heap Sort is useful when consistent performance and low extra memory usage are important.